```
d3.selectAll("g.overallG").select("circle").transition().duration(1000)
              .attr("r", function(p) {
                  return radiusScale(d[datapoint]);
              });
```

Now when we click our buttons, the sizes of the circles change, and the change is also animated. This isn't just for show. We're encoding new data, indicating the change between two datapoints using animation. When there was no animation, the reader had to remember if there was a difference between the ranking in draws and wins for Germany. Now the reader has an animated indication that shows Germany visibly shrink or grow to indicate the difference between these two datapoints.

The use of transitions also allows us to delay the change through the `.delay()` function. Like the `.duration()` function, `.delay()` is set with the wait in milliseconds before implementing the change. Slight delays in the firing of an event from an interaction can be useful to improve the legibility of information visualization, allowing users a moment to reorient themselves to shift from interaction to reading. But long delays will usually be misinterpreted as poor web performance.

Why else would you delay the firing of an animation? Delays can also draw attention to visual elements when they first appear. By making the elements pulse when they arrive onscreen, you let users know that these are dynamic objects and tempt users to click or otherwise interact with them. Delays, like duration, can be dynamically set based on the bound data for each element. You can use delays with another feature: transition chaining. This sets multiple transitions one after another, and each is activated after the last transition has finished. If we amend the code in `overallTeamViz()` that first appends the `<circle>` elements to our `<g>` elements, we can see transitions of the kind that produce the screenshot in figure 3.6:

```
teamG
.append("circle").attr("r", 0)
.transition()
.delay(function(d,i) {return i * 100})
.duration(500)
.attr("r", 40)
.transition()
.duration(500)
.attr("r", 20);
```

This causes a pulse because it uses transition chaining to set one transition, followed by a second after the completion of the first. You start by drawing the circles with a

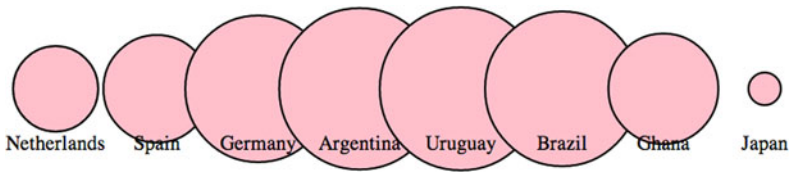


Figure 3.6 A screenshot of your data visualization in the middle of its initial drawing, showing the individual circles growing to an exaggerated size and then shrinking to their final size in the order in which they appear in the bound dataset.